CodeClash demo day

Host `http://129.225.92.29`. Every DEMO password is `codeclash`. Nine steps, 10:40. Step 5 sits outside the six required scenarios and is the first thing to cut.

At	Step		Scenario	Time
0:00	1	Sign up and the onboarding email	End to end	0:40
0:40	2	Cola: two attacks on the sandbox	Security	0:35
1:15	3	Cola: the correct solution	End to end	0:50
2:05	4	Admin promotes the new account	Security	0:25
2:30	5	Organiser: contest and integrity	Extra, cut first	0:40
3:10	6	Setter: Leap year and hardening	Agentic AI	2:00
5:10	7	Two browsers, one seat	Concurrency	2:30
7:40	8	A bad run, then a rejudge	Failure	1:30
9:10	9	Jira, GitHub, test	Traceability	1:30

Before the slot

None of this is done on stage. The ten minutes are only clicks and pastes.

- The host has been reset: DEMO One seat is empty, the integrity cup is there, and no Leap year is left over.
- The onboarding mail has already arrived once in the inbox you will use, including the spam folder.
- `http://129.225.92.29/health` shows `ok`, with Mongo and Redis true.
- Browser A is signed out, on the home page.
- Browser B is signed in as DEMO Student, on the DEMO One seat contest, with a Cola tab ready.
- A private window is signed in as DEMO Rival, on the same DEMO One seat contest.
- `leap-year-fields.txt` is open to copy from, and `leap-year-tests.zip` is visible in Explorer.
- Tabs are open: Jira CC-20, pull request 17 and its CI job, pull request 47.

Accounts

Name	Email	Used for
DEMO Admin	`demo-admin@codeclash.dev`	promotes the new account; audit log
DEMO Setter	`demo-setter@codeclash.dev`	authoring and hardening
DEMO Organiser	`demo-organiser@codeclash.dev`	spare organiser if the live signup fails
DEMO Student	`demo-student@codeclash.dev`	Browser B: takes the one seat, then the bad Cola run
DEMO Rival	`demo-rival@codeclash.dev`	private window: joins the waitlist; also the integrity pair

The live signup becomes the organiser in step 4. A role change takes effect after that account signs out and in again.

The ten minutes

1 Sign up and the onboarding email

End to end · 0:40

Browser A, signed out. A real inbox you tested the day before.

1. Home, create a free account. Name, email, password, create.
2. Open the inbox. The six-digit code arrives.
3. Type the code. You land on Contests, signed in.

Say. An unverified account can read everything but cannot take a seat or submit. This new account is the one we promote later, so watch the name.

See. Mail from CodeClash with a six-digit code within about 20 seconds. The header shows the new name. The nav has contests and problemset only.

If it fails. No mail in 20 seconds: sign in as DEMO Student and carry on. The address can be reused after the slot.

2 Cola: two attacks on the sandbox

Security · 0:35

The new account, or DEMO Student. Files: `demo/cola/1-attack-break-out.py`, `demo/cola/2-attack-hydra.py`.

1. Problemset, Cola, language Python.
2. Open, from the editor toolbar, `1-attack-break-out.py`, submit. Read the verdict card out loud.
3. Open `2-attack-hydra.py`, submit. Read that card too.

Say. Attack 1 makes one request, to the metadata address that would hand over the VM's identity. Attack 2 allocates memory until the container's own cap kills it. Each run is its own container: no network, a read-only root, a 64 MB tmpfs, 64 pids, uid 65534, every capability dropped, no-new-privileges, and a hard kill on timeout.

See. Attack 1 is a runtime error. The card says the metadata request was blocked. Attack 2 is memory limit exceeded. The next submit still runs.

Optional third file, `demo/cola/4-attack-fork.py`: it forks until the 64-process cap refuses. The card says the fork was blocked, and the next submit still runs.

3 Cola: the correct solution

End to end · 0:50

Same account. File: `demo/cola/3-correct.py`.

1. Open `3-correct.py`, run samples, submit.
2. Point at the live verdict card, then the solved count on the profile.

Say. One submit is one transaction: stored in Mongo, put on a Redis stream, claimed by a judge, run in a fresh container, and the verdict pushed back over the socket. A run stops at the first failing test. The timer includes the container start.

See. Accepted, with time and memory per test.

4 Admin promotes the new account

Security · 0:25

The new account first, then DEMO Admin. Do this in the address bar, so the room sees the page change.

1. Still signed in as the new account, type `/control`, then `/authoring`, then `/control/integrity`, then `/admin`. Each one lands back on Contests. The nav has no staff links.
2. Sign out. Sign in as DEMO Admin.
3. Admin, People. Find the new email. Set the role to organiser.
4. Admin, Audit. The new `user.role` row is on top.

Say. Hiding the link is not the guard. Each of those addresses asked the server, and the server refused because the signed token says participant. Changing the token by hand fails the signature, so a forged admin token is rejected too. The role change is a new row in an append-only log. It takes effect when that person signs in again.

See. Four addresses, four bounces back to Contests. The audit row reads `user.role` to organiser.

5 Organiser: create a contest, integrity review

Extra · 0:40

The new account, signed out and back in. The role lives in the token. Cut this step first if you are behind.

1. The nav now shows control and integrity. Control, live rounds: running contests with a clock, a freeze mark, the queue and the leaders.
2. New contest. Pick Absolute difference, not Cola. An open contest holds its problems out of the problemset, and steps 7 and 8 still need Cola. Create and open registration.
3. Integrity, DEMO Integrity cup, run again, click DEMO Student and DEMO Rival.

Say. Rejudge, in the side panel, puts finished submissions back on the queue. The audit row keeps the old verdicts and the reason. Integrity strips comments and strings, turns names into ID and numbers into NUM, takes 5-token fingerprints and winnows them. Code most people share is ignored. A match is a review, never an automatic penalty.

See. One pair at 95 percent, same network, 14 of 20 lines marked on each side. Dismiss or confirm writes the decision with your name and the time.

If it fails. Behind schedule: skip the contest form and show only the pair.

6 Setter: author Leap year and harden its tests

Agentic AI · 2:00

DEMO Setter. Files: `demo/authoring/leap-year-fields.txt`, `demo/authoring/leap-year-tests.zip`.

1. Authoring, new problem. Paste the title, click Private, easy, the tags, the statement, create draft.
2. Statement. Examples: 2024 to Leap, 2023 to Common. Save.
3. Tests. Drag `leap-year-tests.zip` onto the dashed box. It saves first. Spec: `y int 1..9999`.
4. Solutions. Paste the reference and the two wrong solutions. Save.
5. Check and harden. Run the publish check. It is blocked.
6. Run hardening. Talk while the five stage cards light up, about 70 seconds.
7. Click a red tile under Every mutant. Tick both test proposals, approve selected, run the publish check.

Say. The tests skip both century cases on purpose, and the publish check catches that a wrong solution still passes. Hardening plants small bugs in the reference and attacks them with boundary, random and max-size inputs. Gemini plans extra attacks aimed at each survivor. The agent only proposes. Every input passes the spec validator, the

expected output comes from the reference in the sandbox, there is a per-run and a monthly token cap, and nothing changes until the setter approves.

See. The publish check says “wrong solution survived: forgets the century rule”. Fourteen mutants: seven trivial, seven survived, all on the century rule. The score moves from 0 percent to 100 percent after approval. The proposals are 400 to Leap and 100 to Common, each naming the mutant it kills. After approval there are 18 tests, the publish check passes, and the problem stays private.

If it fails. To stay inside two minutes, create the draft and paste through Solutions just before the slot. Live, show the filled steps, then check and harden. If the AI pill is not “available”, the deterministic mutants and attacks still run.

7 Two browsers, one seat

Concurrency · 2:30

Put Browser B and the private window side by side. Both are already on DEMO One seat. The contest has capacity 1 and nobody in it.

1. Browser B, DEMO Student: click **register**. The toast says “You are registered”. The pill says registered. The page says 0 seats left, and the button is gone.
2. Click where the button was, or refresh. There is still one pill and no second seat.
3. Private window, DEMO Rival: click **register**. The toast says “Contest is full: you are #1 on the waitlist”. The pill says waitlist #1.
4. Back to Browser B: click **withdraw** and confirm “Give up your seat? The next person on the waitlist takes it.”
5. Refresh the private window. Rival’s pill now says registered, and Student no longer has a seat.

Say. The seat is one conditional update, reserved less than capacity, inside a transaction. A unique index allows one active seat per person, which is why the button disappears instead of granting a second seat. The waitlist is a sequence, so the position is unique and first in, first out. Withdraw and promote are the same transaction: the seat never goes empty in between. The same rule is the test on pull request 17: 400 users race for 200 seats and get exactly 200, with no duplicates. Under k6, 100 requests a second for 30 seconds, none failed.

See. Student registered and Rival at waitlist #1, at the same time, on screen. After withdraw, Rival is registered and the contest still reads 1 of 1.

If it fails. If Student is already seated from a rehearsal, click withdraw first so the contest is empty, then start this step again.

8 A bad run, then a rejudge

Failure · 1:30

Browser B, problemset, Cola. Practice needs no seat. Then the organiser browser, which is the new account.

1. Select all in the editor and paste this. Submit.

```
print(0)
```

2. Read the verdict card: Wrong answer, and the test it failed.
3. Select all and paste the correct solution. Submit.

```
import sys
n, a, b, c = map(int, sys.stdin.read().split())
ways = 0
for twos in range(min(c, n // 2) + 1):
    rest = n - 2 * twos
    fewest_ones = max(0, rest - a // 2)
    most_ones = min(b, rest)
    if most_ones >= fewest_ones:
        ways += most_ones - fewest_ones + 1
print(ways)
```

4. On the organiser browser: Control, the rejudge panel. Contest: DEMO Integrity cup. Reason: `recomputing after a bad test`. Leave the submission box empty. Click rejudge.
5. Admin, Audit. The `contest.rejudge` row is on top and carries that reason.

Say. The wrong program is a finished verdict, not a stuck judge. The container is thrown away and the next submission is a new job, which is why Accepted follows immediately. Rejudge puts finished submissions back on the same queue. The old verdicts stay in the audit row, so a recomputation does not erase what the judge said the first time. A worker that dies mid-run leaves its message pending, and after 60 seconds another worker claims it. A claim token makes the verdict count once.

See. Wrong answer, then Accepted, on the same problem. The toast says “Queued 6 submissions for rejudge”. The audit row shows the reason.

9 Jira, GitHub, test

Traceability · 1:30

Tabs opened before the slot.

1. Jira CC-20, “US-06 Reserve seat with conditional update + unique index”, Done.
2. Pull request 17, branch `feature/CC-20-seat-reservation`: `registration.ts` and the unique partial index in `indexes.ts`.
3. The pull request’s CI check is green. Open the job log at the seats test.
4. The test: `server/src/domain/domain.test.ts`, seats, “gives exactly 200 seats to 400 distinct users and no duplicates”.
5. Close the loop. That is the rule the two browsers just showed.

Say. Every branch, commit and pull request title carries the Jira key, so the chain is searchable both ways with `git log --grep CC-20`. A second chain, if asked: CC-53, pull request 47 with CI and end-to-end green, then `judge/src/languages.test.ts`.

See. Story, pull request, CI run, named test, live behaviour, in that order.

Part B coverage

Scenario	Required	Planned	Steps
Normal end-to-end transactional workflow	1:30	1:30	1, 3
Agentic AI functionality	2:00	2:00	6
Concurrent access and prevention of double allocation	2:30	2:30	7
Failure handling and recovery	1:30	1:30	8
Security and prevention of unauthorized access	1:00	1:00	2, 4
Jira–GitHub–Test traceability	1:30	1:30	9

Traceability links

- Jira CC-20: `https://cs455-codeclash.atlassian.net/browse/CC-20`
- Pull request 17: `https://github.com/SrujanBhirud/CodeClash/pull/17`
- Its CI job: `https://github.com/SrujanBhirud/CodeClash/actions/runs/39260946917/job/118088124955`
- Jira CC-53: `https://cs455-codeclash.atlassian.net/browse/CC-53`
- Pull request 47: `https://github.com/SrujanBhirud/CodeClash/pull/47`
- Its CI job: `https://github.com/SrujanBhirud/CodeClash/actions/runs/42036285601/job/127275210714`

If the hosted demo is down

Use the copy already running on this PC: the app at `http://127.0.0.1:5173` and the inbox at `http://127.0.0.1:8025`. The same clicks and the same pastes. The accounts and DEMO One seat are on that copy too.